Digital System Design and Implementation

Final Exam (113/06/04)

Spring 2024

Total points: 100 points

- Please **download** the template file DSD.v, which contains all the required I/O in each question and **rename** it as DSDxxxxxxxxx.v or DSDxxxxxxxxx.sv, where x is your student ID, for example, DSD107501001.v.
- Use the template to ensure your module and input as well as output ports having **the same module/port names** as described in the following.
- **Discussion is not allowed, but you can check your own lecture notes.**
- If partial function is correct, you can still get some points. So, try to write codes that can work even that only partial functions are implemented.

In this final exam, we want to process one-dimensional sequence in a given pattern. Please write each module first. The clock signal is called "**Clock**" and the asynchronous active-high reset signal, called "**Reset**" is used for all the questions.

1. Please write a 5-bit decrementer which can count down after reset. Its timing diagram is given as follows. Its output signal is called "**DecOut**". (10%)

   module Decrementer(DecOut, Clock, Reset);

   endmodule

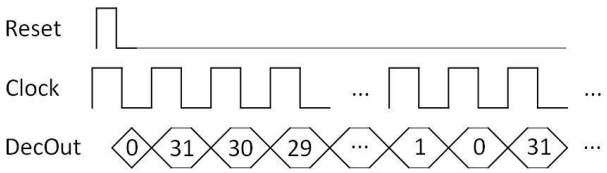


Fig. 1 Timing diagram of a decrementer.

2. Now, extend the function of the module in Q1. We want to construct a module that can be an incrementer or decrementer according to the control signal, "**IorD**" shown in Fig. 2. Hence, it can generate the output by addition or subtraction. In addition, one input signal, called "**Enable**", is used to control the operation of the module. If it is low, then the module will keep its state. (16%)

   module Counting(CountOut, IorD, Enable, Clock, Reset);

   endmodule

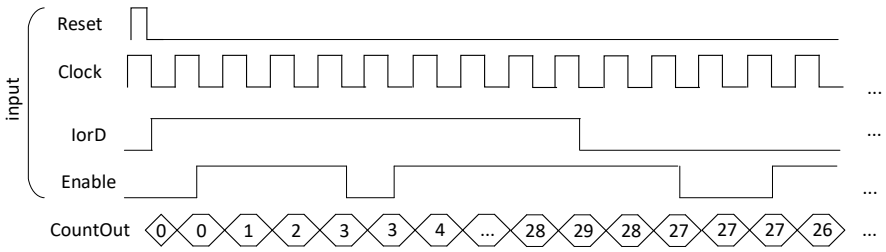
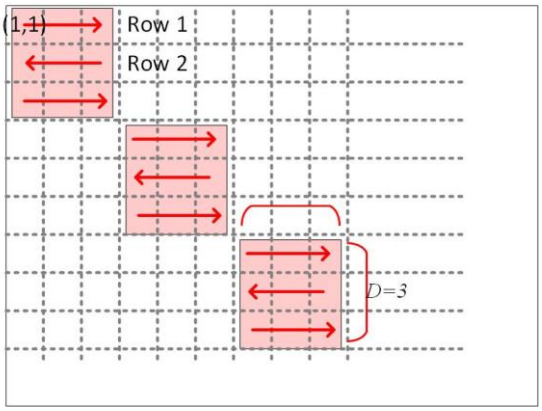


Fig. 2 Timing diagram of the counting module.

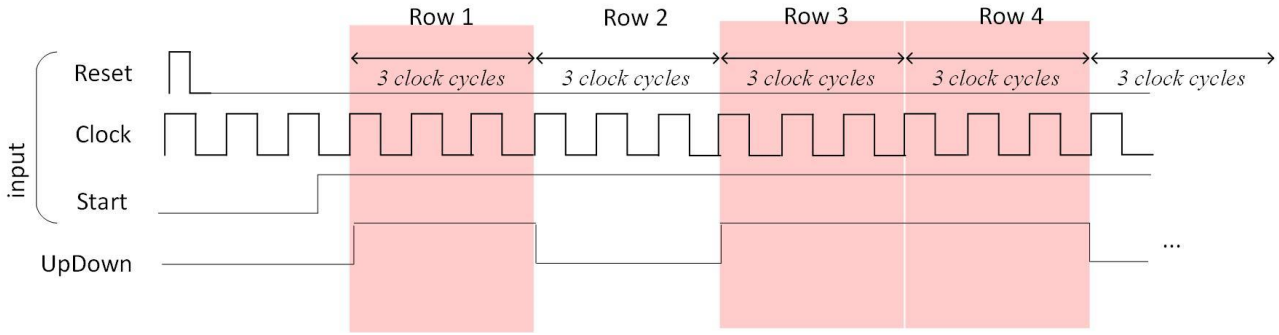
3. We want to generate a special pattern for accessing the elements of a matrix that is stored in a memory array. Because the matrix is block diagonal, which means that the non-zero elements exist along the diagonal and in a square shape as shown in Fig. 3(a), we need to generate the special sequence as shown in Fig. 3(a) to control the module "Counting" in Q2. Assume that the size of the block along the diagonal is 3×3. You need to generate a signal, call "**UpDown**" as shown in Fig. 3(b), where "**Start**" signal triggers the operation. (16%)

   module GenerateUpDownCtr(UpDown, Start, Clock, Reset);

   endmodule



(a)



(b)

Fig. 3 (a) The access pattern in a block-diagonal matrix. (b) The required signal pattern of "**UpDown**".

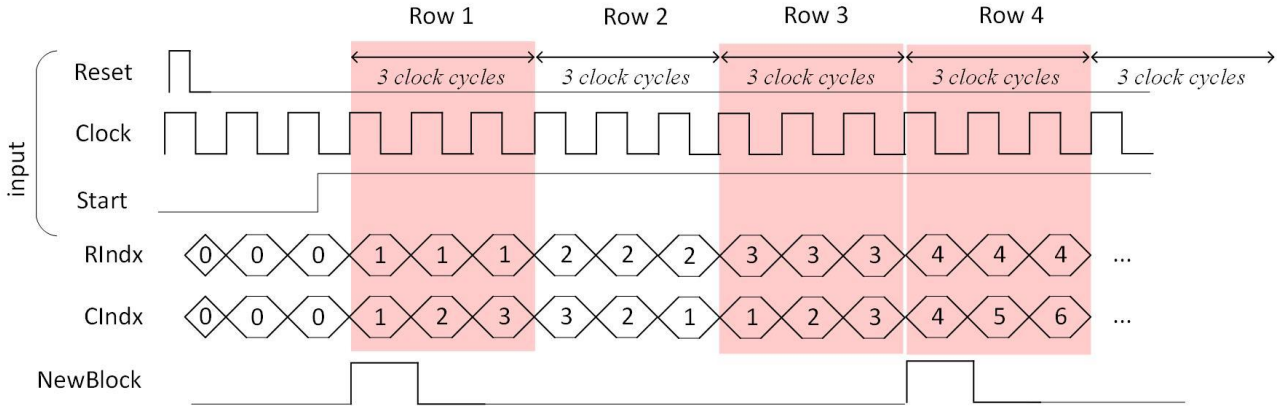
4. Now, we want to generate the row index and column index, counting from 1, to access the elements of the matrix according to the setting of D. Assume that the matrix size along the row or the column is smaller than 64. Hence, 6 bits are reserved for output signals "**RIndx**" and "**CIndx**", corresponding to row index and column index. Note that you need to call module "***Counting***" in Q2 and "***GenerateUpDownCtr***" in Q3. Similarly, "**Start**" signal triggers the operation. The signal "**NewBlock**" asserts when the first element in the diagonal block appears. (We will replace the

correct one when testing. So, don't worry about the correctness of your codes in Q2 and Q3. Just focus on this question given the correct modules in Q2 and Q3.) The timing diagram of the input and output signals is shown in Fig. 4. (24%)

module GenerateRCIndex(NewBlock, RIndx, CIndx, Start, Clock, Reset);

Counting UQ2(.CountOut( ), .IorD( ), .Enable( ), .Clock( ), .Reset( ));
GenerateUpDownCtr UQ3(.UpDown( ), .Start( ), .Clock( ), .Reset( ));

endmodule



5. Given the square matrix size in Q3 is $M \times M$, where M is smaller than 64 and can be set by the user. Write a two-port synchronous read/write register file that can save the content of the whole matrix satisfying the possible settings of M. Port A is used for reading with output "**QA**" and port B is used for writing with input "**DB**". Thus, we don't need to specify write enable signal for port A. The addresses for port A and port B are "**AddA**" and "**AddB**". The signal "**WEB**" is active high. The content of the matrix is expressed as 10 bits. (16%)

module RFTwoPort(QA, AddA, DB, AddB, WEB, Clock);

endmodule

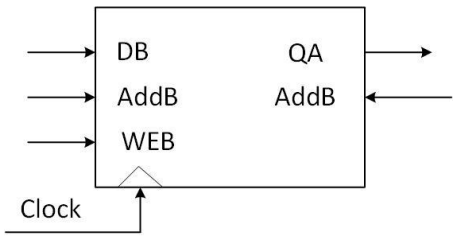


Fig. 5 I/O of module "***RFTwoPort***".

6. Assume that the element of the matrix is expressed in 2's complement and has been saved continuously in the register file described in Q5. The matrix size is $M \times M$, and M is specified by input signal "**SizeM**" and is smaller than 64. For simplicity, $M/3$ is an integer. The (1,1)th element is saved in address 0, and (1,2)th element is saved in address 1 and so on, as shown in

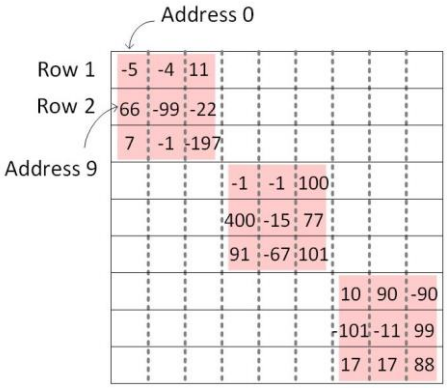
Fig. 6(a). Now, we will read the elements in the diagonal blocks, which can be accessed by the signals of module "***GenerateRCIndex***" in Q4, and then the number of negative elements in each block is calculated. The number of negative elements in the previous block, indicated by signal "**NegNo**" will be updated when signal "**NBlock**" asserts. For example, if $M = 9$, the output of the module "***CalculateNegativeDBlock***" is depicted in Fig. 6(b). Note that signals "**NBlock**", and "**StartCal**" will be assigned to "**NewBlock**", and "**Start**" of the module "***GenerateRCIndex***".

// Ports DPortB, AddBPort, WEBPort of register file are reserved for testing.
// Don't remove them, but declare them.
module CalculateNegativeDBlock(NegNo, NBlock, SizeM, StartCal, Clock, Reset, DPortB, AddBPort, WEBPort);

GenerateRCIndex
UQ4(.NewBlock(NBlock), .RIndx( ), .CIndx( ), .Start(StartCal), .Clock( ), .Reset( ));

RFTwoPort
UQ5(.QA( ), .AddA( ), .DB(DPortB), .AddB(AddBPort), .WEB(WEBPort), .Clock( ));

endmodule



(a)

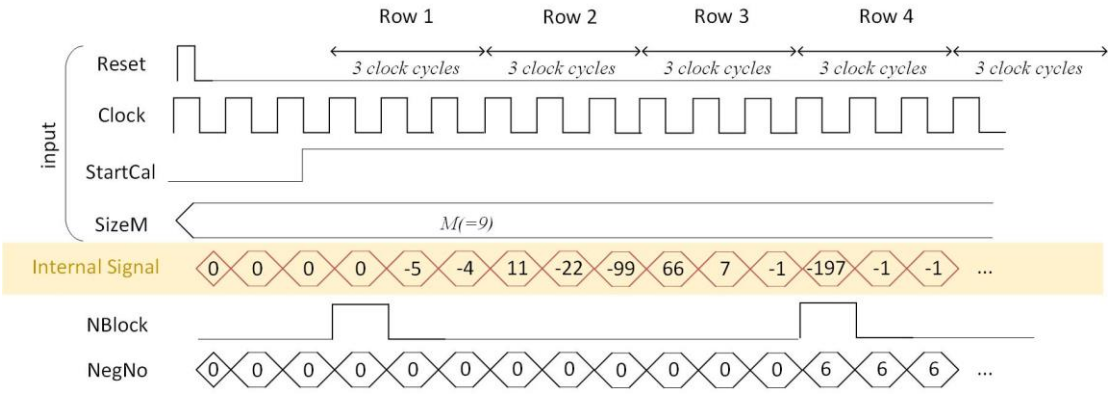


Fig. 6 (a) Example of a block-diagonal matrix. (b) Timing diagram of the module "***CalculateNegativeDBlock***".